

# Frontend: Kubernetes Deployment Guide

Welcome to the Kubernetes deployment configuration for the **Frontend**. This folder ( `k8s/frontend/` ) contains the YAML files that tell a Kubernetes cluster exactly how to run, scale, and route traffic to our React + Nginx application.

Let's break down each file step-by-step so you understand what it does.

## 1. `config.yaml` (The Nginx ConfigMap)

**Purpose:** To inject our custom Nginx routing rules into the container without rebuilding the Docker image.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: frontend-nginx-config
```

- **What is a ConfigMap?** It's a way to store non-secret configuration data (like a configuration file) outside of the application code.

```
data:
  nginx.conf: |
    server {
      listen 80;
      # ... serves our React files ...

      location /api/ {
        proxy_pass http://portfolio-backend-service:8080/api/;
        proxy_set_header Host $host;
      }
    }
```

- **The Secret Sauce:** We tell Nginx to listen on port 80. If someone requests a normal page, Nginx serves the static React files. But if the URL starts with `/api/`, Nginx automatically acts as a **Reverse Proxy** and forwards the request to the `portfolio-backend-service` inside the cluster. This prevents nasty CORS networking errors in the browser!

## 2. `deployment.yaml` (The Application Pods)

**Purpose:** To deploy the actual Docker container holding our React app and manage its lifecycle.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: portfolio-frontend
```

- **What is a Deployment?** It's a Kubernetes boss that manages our pods. If a pod crashes, the Deployment notices and spins up a new one automatically to ensure the app stays online.

```
spec:
  replicas: 1
```

- **Replicas:** We are telling K8s we want exactly 1 copy of our frontend running. If we set this to `3`, K8s would run 3 identical copies for high availability!

```
containers:
  - name: frontend
    image: srtechops/three-tier-k8s-project:frontend-latest
    ports:
      - containerPort: 80
```

- **The Container:** This block specifies the exact Docker image K8s should pull from Docker Hub, and tells K8s the app is listening on port 80 inside the container.

```
volumeMounts:
  - name: nginx-config-volume
    mountPath: /etc/nginx/conf.d/default.conf
    subPath: nginx.conf
```

- **Mounting the ConfigMap:** Here we grab the `frontend-nginx-config` (from `config.yml`) and overwrite Nginx's default configuration file inside the running container.

---

### 3. `service-nodeport.yml` (The Network Exposer)

**Purpose:** To allow external users on the internet (or your local browser) to access the frontend pod.

```
apiVersion: v1
kind: Service
metadata:
  name: portfolio-frontend-service
```

- **What is a Service?** Pod IP addresses change all the time. A Service gives the pods a stable IP, DNS name, and load balancing.

```
spec:
  type: NodePort
  selector:
    app: frontend
```

- **NodePort vs ClusterIP:** By default, Services are `ClusterIP` (internal only). By making it a `NodePort`, Kubernetes opens a specific port on the physical machine (Node) running the cluster, so external traffic can reach in.
- **Selector:** This tells the Service: "Route network traffic to any pod that is labeled `app: frontend`".

```
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30082
```

- `port: 80` : The port the Service listens on.
- `targetPort: 80` : The port the container is listening on.
- `nodePort: 30082` : The external port you type into your browser! (e.g., `http://localhost:30082`).

---

## Summary for Students

Deployments manage **Pods** (the running containers). Services manage **Network Traffic** (how things talk to the pods). ConfigMaps manage **Settings** (so we don't hardcode them into the image).